B20 Prompting and structured output

System prompts, few-shot turns, validated JSON output and model calls you can test and reproduce.

B20-L1 Prompts as data, system prompt, messages and few-shot turns

Instructions in the system prompt, data in tagged user messages, examples as earlier turns, and a version on every result.

```
response = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=1024,
    system=spec.system,
    messages=build_messages(answer, spec.examples),
)
```

The two inputs to an LLM call	A system prompt with instructions, and a list of messages with role user or assistant. The first message is a user turn.
Where respondent text goes	In a user message inside tags such as <answer>...</answer>, with tag text removed from the answer first.
Few-shot examples	Earlier turns: the example answer as a user message, then the wanted label as an assistant message.
What every model result records	The prompt version and the model ID, with the prompt loaded from a versioned file.

B20-L2 Structured output, validate every reply and retry once

Validate every reply against a model of what you need, retry once with the error, and send the rest to review.

```
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field


class ThemeCoding(BaseModel):
    model_config = ConfigDict(extra="forbid")

    theme: Literal["Price", "Service", "Product", "Other"]
    confidence: float = Field(ge=0, le=1)


coding = ThemeCoding.model_validate_json(reply_text)
```

Parse and validate a model reply in one step	Model.model_validate_json(reply). Broken JSON and bad fields both raise ValidationError.
Structured output in anthropic 2.4.0	client.messages.parse(..., output_format=PydanticModel), then response.parsed_output.
When a reply fails validation	Add the reply as an assistant turn and the error as a user turn, retry, and stop after a fixed number of attempts.
When every attempt fails	Store needs_review with the raw reply. Never guess a label.

B20-L3 Model calls in a pipeline, stop reasons, caching and run logs

Inject the model, reject replies that did not finish, cache on model plus prompt version plus messages, and log every call.

```
text = json.dumps({"model": model, "prompt_version": version, "messages": messages},
  sort_keys=True)
key = hashlib.sha256(text.encode()).hexdigest()
```

stop_reason values to reject	Anything but end_turn: max_tokens means the reply was cut off, refusal means the model declined.
Cache key for model replies	sha256 of json.dumps({model, prompt_version, messages}, sort_keys=True), as hexdigest().
What the anthropic 2.4.0 SDK retries	Connection errors and status 408, 409, 429 and 500 or above, twice by default (max_retries=2).
One run-log record per model call	run_id, UTC timestamp, model, prompt version, cache hit, stop reason, input and output tokens.